

El SAT y el vacío de la intersección de una gramática de concatenación de rango con un lenguaje regular finito

Luis Alejandro Arteaga Morales

Facultad de Matemática y Computación
Universidad de La Habana

2026

Tutores: MSc. Fernando Raúl Rodríguez Flores
Lic. Raudel Alejandro Gómez Molina

El problema SAT

- Determinar si una fórmula booleana es satisfacible.

El problema SAT

- Determinar si una fórmula booleana es satisfacible.
- Primer problema demostrado NP-completo.

El problema SAT

- Determinar si una fórmula booleana es satisfacible.
- Primer problema demostrado NP-completo.
- Aplicaciones: criptografía, circuitos, CSP, programación lógica.

El problema SAT

- Determinar si una fórmula booleana es satisfacible.
- Primer problema demostrado NP-completo.
- Aplicaciones: criptografía, circuitos, CSP, programación lógica.
- Interesa delimitar fragmentos polinomiales.

El problema SAT

- Determinar si una fórmula booleana es satisfacible.
- Primer problema demostrado NP-completo.
- Aplicaciones: criptografía, circuitos, CSP, programación lógica.
- Interesa delimitar fragmentos polinomiales.
- Resolverlo eficiente $\Rightarrow P = NP$.

- En MATCOM se estudia el SAT con lenguajes formales.

Enfoque basado en lenguajes formales

- En MATCOM se estudia el SAT con lenguajes formales.
- A cada fórmula se asocia un autómata y una gramática.

Enfoque basado en lenguajes formales

- En MATCOM se estudia el SAT con lenguajes formales.
- A cada fórmula se asocia un autómata y una gramática.
- Autómata: finitas interpretaciones, ocurrencias por separado.

Enfoque basado en lenguajes formales

- En MATCOM se estudia el SAT con lenguajes formales.
- A cada fórmula se asocia un autómata y una gramática.
- Autómata: finitas interpretaciones, ocurrencias por separado.
- Gramática: consistencia entre ocurrencias.

Enfoque basado en lenguajes formales

- En MATCOM se estudia el SAT con lenguajes formales.
- A cada fórmula se asocia un autómata y una gramática.
- Autómata: finitas interpretaciones, ocurrencias por separado.
- Gramática: consistencia entre ocurrencias.
- **Satisfacible $\Leftrightarrow$ intersección no vacía.**

- En MATCOM se estudia el SAT con lenguajes formales.
- A cada fórmula se asocia un autómata y una gramática.
- Autómata: finitas interpretaciones, ocurrencias por separado.
- Gramática: consistencia entre ocurrencias.
- **Satisfacible $\Leftrightarrow$ intersección no vacía.**
- Antecedentes en MATCOM: CFG y sRCG.

- En MATCOM se estudia el SAT con lenguajes formales.
- A cada fórmula se asocia un autómata y una gramática.
- Autómata: finitas interpretaciones, ocurrencias por separado.
- Gramática: consistencia entre ocurrencias.
- **Satisfacible $\Leftrightarrow$ intersección no vacía.**
- Antecedentes en MATCOM: CFG y sRCG.
- En paralelo se desarrolló un trabajo con pMCFG.

¿Cómo ayudaría a resolver el SAT?

Si un formalismo permitiera:

¿Cómo ayudaría a resolver el SAT?

Si un formalismo permitiera:

- representar la intersección con el autómata,

¿Cómo ayudaría a resolver el SAT?

Si un formalismo permitiera:

- representar la intersección con el autómata,
- decidir su vacío en tiempo polinomial,

¿Cómo ayudaría a resolver el SAT?

Si un formalismo permitiera:

- representar la intersección con el autómata,
- decidir su vacío en tiempo polinomial,

entonces el SAT se resolvería en tiempo polinomial.

Las RCG: el formalismo de este trabajo

- Antecedentes: vacío polinomial, pero solo ciertas fórmulas.

Las RCG: el formalismo de este trabajo

- Antecedentes: vacío polinomial, pero solo ciertas fórmulas.
- **Este trabajo toma las RCG.**

Las RCG: el formalismo de este trabajo

- Antecedentes: vacío polinomial, pero solo ciertas fórmulas.
- **Este trabajo toma las RCG.**
- Expresan la consistencia de cualquier fórmula.

Las RCG: el formalismo de este trabajo

- Antecedentes: vacío polinomial, pero solo ciertas fórmulas.
- **Este trabajo toma las RCG.**
- Expresan la consistencia de cualquier fórmula.
- Son cerradas bajo intersección.

Las RCG: el formalismo de este trabajo

- Antecedentes: vacío polinomial, pero solo ciertas fórmulas.
- **Este trabajo toma las RCG.**
- Expresan la consistencia de cualquier fórmula.
- Son cerradas bajo intersección.
- **Pero su vacío deja de ser tratable.**

Las RCG: el formalismo de este trabajo

- Antecedentes: vacío polinomial, pero solo ciertas fórmulas.
- **Este trabajo toma las RCG.**
- Expresan la consistencia de cualquier fórmula.
- Son cerradas bajo intersección.
- **Pero su vacío deja de ser tratable.**
- NP-completo: algoritmo polinomial $\Rightarrow P = NP$.

Las RCG: el formalismo de este trabajo

- Antecedentes: vacío polinomial, pero solo ciertas fórmulas.
- **Este trabajo toma las RCG.**
- Expresan la consistencia de cualquier fórmula.
- Son cerradas bajo intersección.
- **Pero su vacío deja de ser tratable.**
- NP-completo: algoritmo polinomial $\Rightarrow P = NP$.
- Subclases con vacío polinomial.

Objetivo general

Objetivo general

Estudiar el vacío de la intersección de una RCG con un lenguaje regular finito como marco para el SAT, y construir una representación de esa intersección.

Objetivos específicos

- Estudiar las RCG y los antecedentes del SAT.

Objetivos específicos

- Estudiar las RCG y los antecedentes del SAT.
- Diseñar un algoritmo que decida ese vacío y caracterizar su costo.

Objetivos específicos

- Estudiar las RCG y los antecedentes del SAT.
- Diseñar un algoritmo que decida ese vacío y caracterizar su costo.
- Modelar el SAT como ese vacío con una RCG de consistencia.

Objetivos específicos

- Estudiar las RCG y los antecedentes del SAT.
- Diseñar un algoritmo que decida ese vacío y caracterizar su costo.
- Modelar el SAT como ese vacío con una RCG de consistencia.
- Construir la gramática producto y demostrarla.

Estructura de la presentación

Preliminares

- Teoría de lenguajes
- Definición de SAT
- Autómata booleano

Gramáticas de concatenación de rango

- Nociones básicas
- Ejemplo de reconocimiento
- Por qué las RCG

Resolver el SAT por enumeración

- Algoritmo por enumeración
- Gramática de consistencia

Construir la gramática producto

- Predicados indexados
- Misma variable, mismo rango
- Ejemplo de reconocimiento
- Correctitud y tamaño

Estructura de la presentación

Preliminares

- Teoría de lenguajes
- Definición de SAT
- Autómata booleano

Gramáticas de concatenación de rango

- Nociones básicas
- Ejemplo de reconocimiento
- Por qué las RCG

Resolver el SAT por enumeración

- Algoritmo por enumeración
- Gramática de consistencia

Construir la gramática producto

- Predicados indexados
- Misma variable, mismo rango
- Ejemplo de reconocimiento
- Correctitud y tamaño

- **Alfabeto** (Σ): conjunto finito de símbolos.

$$\Sigma = \{0, 1\}$$

Teoría de lenguajes

- **Alfabeto** (Σ): conjunto finito de símbolos.

$$\Sigma = \{0, 1\}$$

- **Cadena**: secuencia finita de símbolos de Σ .

$$0110 \in \Sigma^* \quad \varepsilon \text{ (cadena vacía)}$$

- **Alfabeto** (Σ): conjunto finito de símbolos.

$$\Sigma = \{0, 1\}$$

- **Cadena**: secuencia finita de símbolos de Σ .

$$0110 \in \Sigma^* \quad \varepsilon \text{ (cadena vacía)}$$

- **Concatenación**:

$$w_1 = 01 \text{ y } w_2 = 10 \implies w_1 w_2 = 0110$$

- **Alfabeto** (Σ): conjunto finito de símbolos.

$$\Sigma = \{0, 1\}$$

- **Cadena**: secuencia finita de símbolos de Σ .

$$0110 \in \Sigma^* \quad \varepsilon \text{ (cadena vacía)}$$

- **Concatenación**:

$$w_1 = 01 \text{ y } w_2 = 10 \implies w_1 w_2 = 0110$$

- **Lenguaje**: conjunto de cadenas.

$$L = \{0^n 1^n \mid n \geq 0\}$$

- **Alfabeto** (Σ): conjunto finito de símbolos.

$$\Sigma = \{0, 1\}$$

- **Cadena**: secuencia finita de símbolos de Σ .

$$0110 \in \Sigma^* \quad \varepsilon \text{ (cadena vacía)}$$

- **Concatenación**:

$$w_1 = 01 \text{ y } w_2 = 10 \implies w_1 w_2 = 0110$$

- **Lenguaje**: conjunto de cadenas.

$$L = \{0^n 1^n \mid n \geq 0\}$$

- **Formalismo**: mecanismo de algún tipo para describir lenguajes.

Problema de la palabra

Determinar si una cadena pertenece a un lenguaje. ($w \in L?$)

Problemas de interés

Problema de la palabra

Determinar si una cadena pertenece a un lenguaje. ($w \in L?$)

Problema del vacío

Determinar si un lenguaje es vacío. ($L = \emptyset?$)

Problemas de interés

Problema de la palabra

Determinar si una cadena pertenece a un lenguaje. ($w \in L?$)

Problema del vacío

Determinar si un lenguaje es vacío. ($L = \emptyset?$)

El SAT se decide por el problema del vacío.

Definición. Genera cadenas con reglas de producción.

$$G = (N, \Sigma, P, S)$$

- N : no terminales
- Σ : terminales
- P : producciones
- S : símbolo inicial

Definición. Genera cadenas con reglas de producción.

$$G = (N, \Sigma, P, S)$$

- N : no terminales
- Σ : terminales
- P : producciones
- S : símbolo inicial

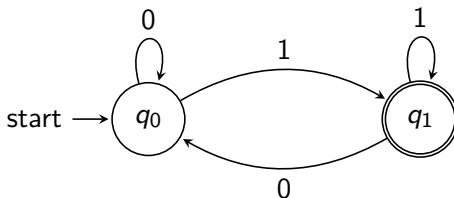
$$S \rightarrow 0S1 \mid 01 \quad L(G) = \{0^n 1^n \mid n \geq 1\}$$

Definición. Reconoce cadenas recorriendo estados.

$$A = (Q, \Sigma, \delta, q_0, F)$$

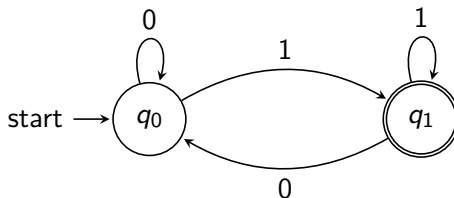
Definición. Reconoce cadenas recorriendo estados.

$$A = (Q, \Sigma, \delta, q_0, F)$$



Definición. Reconoce cadenas recorriendo estados.

$$A = (Q, \Sigma, \delta, q_0, F)$$



Acepta las cadenas que terminan en 1: **101** se acepta, 100 se rechaza.

Definición de SAT

- **Variable booleana:** toma verdadero o falso.

Definición de SAT

- **Variable booleana:** toma verdadero o falso.
- **Literal:** una variable o su negación (x_1 , $\neg x_1$).

Definición de SAT

- **Variable booleana:** toma verdadero o falso.
- **Literal:** una variable o su negación (x_1 , $\neg x_1$).
- **Cláusula:** disyunción de literales ($x_1 \vee \neg x_2 \vee x_3$).

Definición de SAT

- **Variable booleana:** toma verdadero o falso.
- **Literal:** una variable o su negación ($x_1, \neg x_1$).
- **Cláusula:** disyunción de literales ($x_1 \vee \neg x_2 \vee x_3$).
- **CNF:** conjunción de cláusulas ($(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3)$).

Definición de SAT

- **Variable booleana:** toma verdadero o falso.
- **Literal:** una variable o su negación (x_1 , $\neg x_1$).
- **Cláusula:** disyunción de literales ($x_1 \vee \neg x_2 \vee x_3$).
- **CNF:** conjunción de cláusulas ($(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3)$).
- **Subfórmula:** una parte de la fórmula ($x_2 \vee x_1$).

Definición de SAT

- **Variable booleana:** toma verdadero o falso.
- **Literal:** una variable o su negación (x_1 , $\neg x_1$).
- **Cláusula:** disyunción de literales ($x_1 \vee \neg x_2 \vee x_3$).
- **CNF:** conjunción de cláusulas ($(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3)$).
- **Subfórmula:** una parte de la fórmula ($x_2 \vee x_1$).
- Ejemplo: $x_1 \wedge (x_2 \vee x_1)$.

Definición de SAT

- **Variable booleana:** toma verdadero o falso.
- **Literal:** una variable o su negación (x_1 , $\neg x_1$).
- **Cláusula:** disyunción de literales ($x_1 \vee \neg x_2 \vee x_3$).
- **CNF:** conjunción de cláusulas ($(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3)$).
- **Subfórmula:** una parte de la fórmula ($x_2 \vee x_1$).
- Ejemplo: $x_1 \wedge (x_2 \vee x_1)$.
- Secuencia de ocurrencias: $x_1 x_2 x_1$ ($n = 3$).

Autómata booleano: una ocurrencia

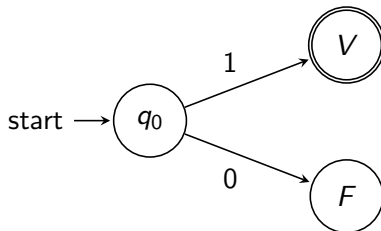
- Cada ocurrencia aporta un símbolo: 1 verdadero, 0 falso.

Autómata booleano: una ocurrencia

- Cada ocurrencia aporta un símbolo: 1 verdadero, 0 falso.
- V acepta (verdadero); F rechaza (falso).

Autómata booleano: una ocurrencia

- Cada ocurrencia aporta un símbolo: 1 verdadero, 0 falso.
- V acepta (verdadero); F rechaza (falso).



Autómata booleano: negación

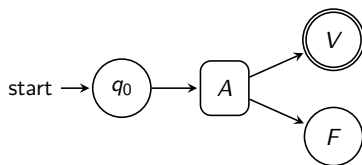
- El autómata de una fórmula compuesta combina los de sus subfórmulas.

Autómata booleano: negación

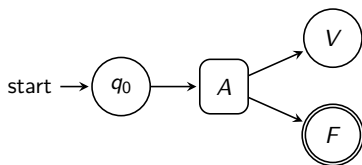
- El autómata de una fórmula compuesta combina los de sus subfórmulas.
- La negación intercambia V y F .

Autómata booleano: negación

- El autómata de una fórmula compuesta combina los de sus subfórmulas.
- La negación intercambia V y F .



(a) f



(b) $\neg f$

Autómata booleano: disyunción

- Disyunción $f_1 \vee f_2$ de dos subfórmulas; A_1, A_2 sus autómatas.

Autómata booleano: disyunción

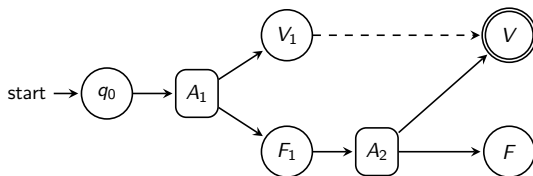
- Disyunción $f_1 \vee f_2$ de dos subfórmulas; A_1, A_2 sus autómatas.
- Si f_1 es verdadera, ya acepta: un camino de extensión lleva a V .

Autómata booleano: disyunción

- Disyunción $f_1 \vee f_2$ de dos subfórmulas; A_1, A_2 sus autómatas.
- Si f_1 es verdadera, ya acepta: un camino de extensión lleva a V .
- Si f_1 es falsa, el valor depende de f_2 .

Autómata booleano: disyunción

- Disyunción $f_1 \vee f_2$ de dos subfórmulas; A_1, A_2 sus autómatas.
- Si f_1 es verdadera, ya acepta: un camino de extensión lleva a V .
- Si f_1 es falsa, el valor depende de f_2 .



Autómata booleano: conjunción

- Conjunción $f_1 \wedge f_2$ de dos subfórmulas; A_1, A_2 sus autómatas.

Autómata booleano: conjunción

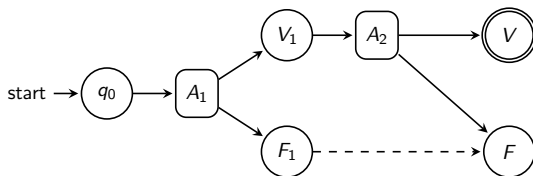
- Conjunción $f_1 \wedge f_2$ de dos subfórmulas; A_1, A_2 sus autómatas.
- Si f_1 es falsa, ya rechaza: un camino de extensión lleva a F .

Autómata booleano: conjunción

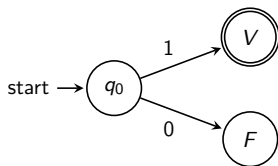
- Conjunción $f_1 \wedge f_2$ de dos subfórmulas; A_1, A_2 sus autómatas.
- Si f_1 es falsa, ya rechaza: un camino de extensión lleva a F .
- Si f_1 es verdadera, el valor depende de f_2 .

Autómata booleano: conjunción

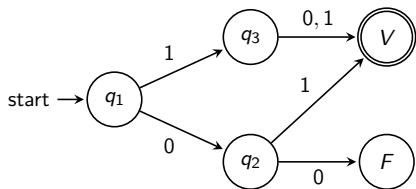
- Conjunción $f_1 \wedge f_2$ de dos subfórmulas; A_1, A_2 sus autómatas.
- Si f_1 es falsa, ya rechaza: un camino de extensión lleva a F .
- Si f_1 es verdadera, el valor depende de f_2 .



Autómata booleano: las subfórmulas



(a) x_1



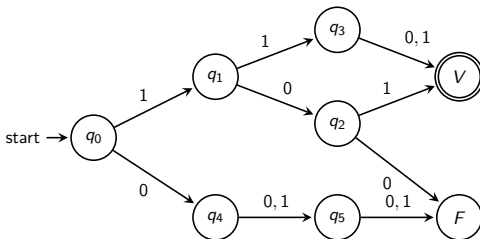
(b) $x_2 \vee x_1$

Autómata booleano de $x_1 \wedge (x_2 \vee x_1)$

- Conjunción: si x_1 es falsa, rechaza; si no, sigue en $x_2 \vee x_1$.

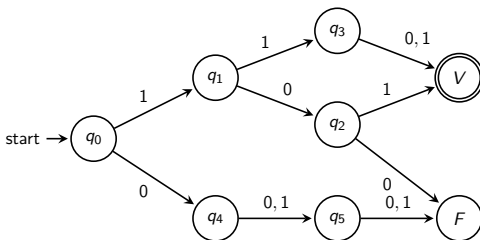
Autómata booleano de $x_1 \wedge (x_2 \vee x_1)$

- Conjunción: si x_1 es falsa, rechaza; si no, sigue en $x_2 \vee x_1$.



Autómata booleano de $x_1 \wedge (x_2 \vee x_1)$

- Conjunción: si x_1 es falsa, rechaza; si no, sigue en $x_2 \vee x_1$.



$$L(A) = \{101, 110, 111\}.$$

Estructura de la presentación

Preliminares

- Teoría de lenguajes
- Definición de SAT
- Autómata booleano

Gramáticas de concatenación de rango

- Nociones básicas
- Ejemplo de reconocimiento
- Por qué las RCG

Resolver el SAT por enumeración

- Algoritmo por enumeración
- Gramática de consistencia

Construir la gramática producto

- Predicados indexados
- Misma variable, mismo rango
- Ejemplo de reconocimiento
- Correctitud y tamaño

Gramáticas de concatenación de rango

- **Predicado:** no terminal con argumentos.

$$A(X, Y)$$

Gramáticas de concatenación de rango

- **Predicado:** no terminal con argumentos.

$$A(X, Y)$$

- **Aridad:** cantidad de argumentos.

$A(X, Y)$ tiene aridad 2

Gramáticas de concatenación de rango

- **Predicado:** no terminal con argumentos.

$$A(X, Y)$$

- **Aridad:** cantidad de argumentos.

$$A(X, Y) \text{ tiene aridad } 2$$

- **Rango:** par de posiciones que delimita una subcadena.

$$\text{en } 0110, (1, 3) = 11$$

Gramáticas de concatenación de rango

- **Predicado:** no terminal con argumentos.

$$A(X, Y)$$

- **Aridad:** cantidad de argumentos.

$$A(X, Y) \text{ tiene aridad } 2$$

- **Rango:** par de posiciones que delimita una subcadena.

$$\text{en } 0110, (1, 3) = 11$$

- **Cláusula:** producción de la gramática.

$$A(XY) \rightarrow B(X) C(Y)$$

Gramáticas de concatenación de rango

- **Predicado:** no terminal con argumentos.

$$A(X, Y)$$

- **Aridad:** cantidad de argumentos.

$$A(X, Y) \text{ tiene aridad } 2$$

- **Rango:** par de posiciones que delimita una subcadena.

$$\text{en } 0110, (1, 3) = 11$$

- **Cláusula:** producción de la gramática.

$$A(XY) \rightarrow B(X) C(Y)$$

- Un predicado **reconoce** un vector de cadenas (dimensión = aridad).

Ejemplo de reconocimiento

- Reconoce $\{ww \mid w \in \{0,1\}^*\}$.

Ejemplo de reconocimiento

- Reconoce $\{ww \mid w \in \{0,1\}^*\}$.

$$S(XY) \rightarrow A(X, Y) \qquad A(0X, 0Y) \rightarrow A(X, Y)$$

$$A(\varepsilon, \varepsilon) \rightarrow \varepsilon \qquad A(1X, 1Y) \rightarrow A(X, Y)$$

Ejemplo de reconocimiento

- Reconoce $\{ww \mid w \in \{0,1\}^*\}$.

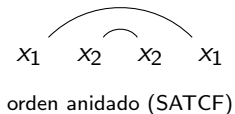
$$S(XY) \rightarrow A(X, Y) \quad A(0X, 0Y) \rightarrow A(X, Y)$$

$$A(\varepsilon, \varepsilon) \rightarrow \varepsilon \quad A(1X, 1Y) \rightarrow A(X, Y)$$

$$S(1010) \rightarrow A(10, 10) \rightarrow A(0, 0) \rightarrow A(\varepsilon, \varepsilon) \rightarrow \varepsilon$$

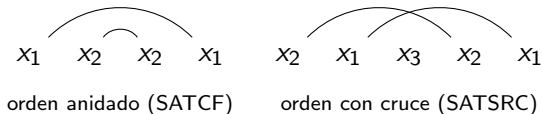
Por qué las RCG

- SATCF (CFG): orden anidado (pila).



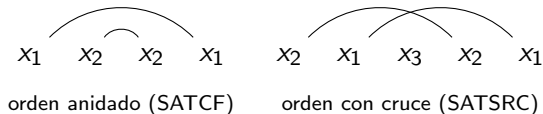
Por qué las RCG

- SATCF (CFG): orden anidado (pila).
- SATSRC (sRCG): admite cruces.



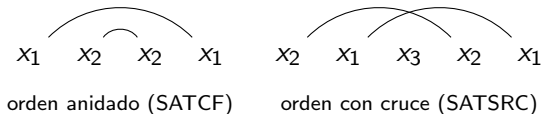
Por qué las RCG

- SATCF (CFG): orden anidado (pila).
- SATSRC (sRCG): admite cruces.
- **La aridad crece con los cruces.**



Por qué las RCG

- SATCF (CFG): orden anidado (pila).
- SATSRC (sRCG): admite cruces.
- **La aridad crece con los cruces.**
- **El vacío se vuelve exponencial.**



Por qué las RCG

- Buscamos resolver más fórmulas (o todas) en tiempo polinomial.

Por qué las RCG

- Buscamos resolver más fórmulas (o todas) en tiempo polinomial.
- Extensión natural de SATSRC: las RCG ($\text{sRCG} \subset \text{RCG}$).

Por qué las RCG

- Buscamos resolver más fórmulas (o todas) en tiempo polinomial.
- Extensión natural de SATSRC: las RCG ($\text{sRCG} \subset \text{RCG}$).
- Abarcar más fórmulas exige no linealidad: $L(\text{RCG lineal}) = L(\text{sRCG})$.

Por qué las RCG

- Buscamos resolver más fórmulas (o todas) en tiempo polinomial.
- Extensión natural de SATSRC: las RCG ($\text{sRCG} \subset \text{RCG}$).
- Abarcar más fórmulas exige no linealidad: $L(\text{RCG lineal}) = L(\text{sRCG})$.
- La no linealidad evita que la aridez crezca con los cruces.

Por qué las RCG

- Buscamos resolver más fórmulas (o todas) en tiempo polinomial.
- Extensión natural de SATSRC: las RCG ($\text{sRCG} \subset \text{RCG}$).
- Abarcar más fórmulas exige no linealidad: $L(\text{RCG lineal}) = L(\text{sRCG})$.
- La no linealidad evita que la aridez crezca con los cruces.
- **Pero su vacío de la intersección es NP-completo** (regular finito).

Por qué las RCG

- Buscamos resolver más fórmulas (o todas) en tiempo polinomial.
- Extensión natural de SATSRC: las RCG ($\text{sRCG} \subset \text{RCG}$).
- Abarcar más fórmulas exige no linealidad: $L(\text{RCG lineal}) = L(\text{sRCG})$.
- La no linealidad evita que la aridez crezca con los cruces.
- **Pero su vacío de la intersección es NP-completo** (regular finito).
- No se sabe si es polinomial; quizá una subclase lo sea.

Por qué las RCG

- Buscamos resolver más fórmulas (o todas) en tiempo polinomial.
- Extensión natural de SATSRC: las RCG ($\text{sRCG} \subset \text{RCG}$).
- Abarcar más fórmulas exige no linealidad: $L(\text{RCG lineal}) = L(\text{sRCG})$.
- La no linealidad evita que la aridez crezca con los cruces.
- **Pero su vacío de la intersección es NP-completo** (regular finito).
- No se sabe si es polinomial; quizá una subclase lo sea.
- Esa subclase no se conoce; el trabajo abre camino a hallarla o descartarla.

Estructura de la presentación

Preliminares

- Teoría de lenguajes
- Definición de SAT
- Autómata booleano

Gramáticas de concatenación de rango

- Nociones básicas
- Ejemplo de reconocimiento
- Por qué las RCG

Resolver el SAT por enumeración

- Algoritmo por enumeración
- Gramática de consistencia

Construir la gramática producto

- Predicados indexados
- Misma variable, mismo rango
- Ejemplo de reconocimiento
- Correctitud y tamaño

Algoritmo de enumeración

- Entrada: una RCG G y un autómata acíclico A .

Algoritmo de enumeración

- Entrada: una RCG G y un autómata acíclico A .
- ¿ $L(G) \cap L(A) \neq \emptyset$? (A acíclico $\Rightarrow L(A)$ finito).

Algoritmo de enumeración

- Entrada: una RCG G y un autómata acíclico A .
- ¿ $L(G) \cap L(A) \neq \emptyset$? (A acíclico $\Rightarrow L(A)$ finito).
- Conservar solo los estados útiles.

Algoritmo de enumeración

- Entrada: una RCG G y un autómata acíclico A .
- ¿ $L(G) \cap L(A) \neq \emptyset$? (A acíclico $\Rightarrow L(A)$ finito).
- Conservar solo los estados útiles.
- Recorrer las cadenas aceptadas por A .

Algoritmo de enumeración

- Entrada: una RCG G y un autómata acíclico A .
- ¿ $L(G) \cap L(A) \neq \emptyset$? (A acíclico $\Rightarrow L(A)$ finito).
- Conservar solo los estados útiles.
- Recorrer las cadenas aceptadas por A .
- Reconocer cada una con G (polinomial).

Algoritmo de enumeración

- Entrada: una RCG G y un autómata acíclico A .
- ¿ $L(G) \cap L(A) \neq \emptyset$? (A acíclico $\Rightarrow L(A)$ finito).
- Conservar solo los estados útiles.
- Recorrer las cadenas aceptadas por A .
- Reconocer cada una con G (polinomial).
- Costo $O(|A| + p(A) R_G)$, con $p(A) = |L(A)|$.

Algoritmo de enumeración

- Entrada: una RCG G y un autómata acíclico A .
- ¿ $L(G) \cap L(A) \neq \emptyset$? (A acíclico $\Rightarrow L(A)$ finito).
- Conservar solo los estados útiles.
- Recorrer las cadenas aceptadas por A .
- Reconocer cada una con G (polinomial).
- Costo $O(|A| + p(A) R_G)$, con $p(A) = |L(A)|$.
- **Exponencial: motiva un solo objeto.**

- Predicados S , Eq_v , L_j ; todos de aridad uno.

- Predicados S , Eq_v , L_j ; todos de aridad uno.
- **Cláusula no lineal:** $S(X) \rightarrow Eq_{v_1}(X) \cdots Eq_{v_m}(X)$.

- Predicados S , Eq_v , L_j ; todos de aridad uno.
- **Cláusula no lineal:** $S(X) \rightarrow Eq_{v_1}(X) \cdots Eq_{v_m}(X)$.
- Eq_v : mismo símbolo en las posiciones de v .

$$Eq_v(W_0 \mathbin{b} W_1 \cdots \mathbin{b} W_r) \rightarrow L_{g_0}(W_0) \cdots L_{g_r}(W_r) \quad (b \in \{0, 1\})$$

- Predicados S , Eq_v , L_j ; todos de aridad uno.
- **Cláusula no lineal:** $S(X) \rightarrow Eq_{v_1}(X) \cdots Eq_{v_m}(X)$.
- Eq_v : mismo símbolo en las posiciones de v .

$$Eq_v(W_0 \mathbin{b} W_1 \cdots \mathbin{b} W_r) \rightarrow L_{g_0}(W_0) \cdots L_{g_r}(W_r) \quad (b \in \{0, 1\})$$

- L_j : cadenas de longitud j .

$$L_j(cX) \rightarrow L_{j-1}(X), \quad L_0(\varepsilon) \rightarrow \varepsilon \quad (c \in \{0, 1\})$$

- Predicados S , Eq_v , L_j ; todos de aridad uno.
- **Cláusula no lineal:** $S(X) \rightarrow Eq_{v_1}(X) \cdots Eq_{v_m}(X)$.
- Eq_v : mismo símbolo en las posiciones de v .

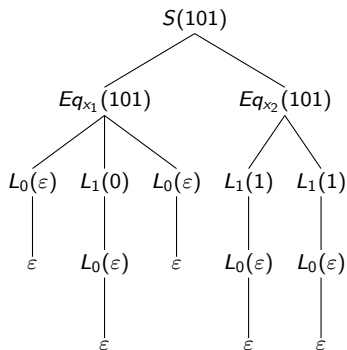
$$Eq_v(W_0 \mathbin{b} W_1 \cdots \mathbin{b} W_r) \rightarrow L_{g_0}(W_0) \cdots L_{g_r}(W_r) \quad (b \in \{0, 1\})$$

- L_j : cadenas de longitud j .

$$L_j(cX) \rightarrow L_{j-1}(X), \quad L_0(\varepsilon) \rightarrow \varepsilon \quad (c \in \{0, 1\})$$

- **Aridad uno para cualquier fórmula.**

101 es consistente: derivación en G_{cons}



110 y 000 no son consistentes.

Estructura de la presentación

Preliminares

- Teoría de lenguajes
- Definición de SAT
- Autómata booleano

Gramáticas de concatenación de rango

- Nociones básicas
- Ejemplo de reconocimiento
- Por qué las RCG

Resolver el SAT por enumeración

- Algoritmo por enumeración
- Gramática de consistencia

Construir la gramática producto

- Predicados indexados
- Misma variable, mismo rango
- Ejemplo de reconocimiento
- Correctitud y tamaño

De la enumeración a un solo objeto

- La enumeración decide el vacío, pero es exponencial.

De la enumeración a un solo objeto

- La enumeración decide el vacío, pero es exponencial.
- Ya resolvemos el SAT para cualquier fórmula, pero no en tiempo polinomial.

De la enumeración a un solo objeto

- La enumeración decide el vacío, pero es exponencial.
- Ya resolvemos el SAT para cualquier fórmula, pero no en tiempo polinomial.
- Y lo hace recorriendo los dos formalismos por separado.

De la enumeración a un solo objeto

- La enumeración decide el vacío, pero es exponencial.
- Ya resolvemos el SAT para cualquier fórmula, pero no en tiempo polinomial.
- Y lo hace recorriendo los dos formalismos por separado.
- Queremos un solo objeto que exprese la intersección.

De la enumeración a un solo objeto

- La enumeración decide el vacío, pero es exponencial.
- Ya resolvemos el SAT para cualquier fórmula, pero no en tiempo polinomial.
- Y lo hace recorriendo los dos formalismos por separado.
- Queremos un solo objeto que exprese la intersección.
- **La gramática producto G' : una RCG con $L(G') = L(G) \cap L(A)$.**

De la enumeración a un solo objeto

- La enumeración decide el vacío, pero es exponencial.
- Ya resolvemos el SAT para cualquier fórmula, pero no en tiempo polinomial.
- Y lo hace recorriendo los dos formalismos por separado.
- Queremos un solo objeto que exprese la intersección.
- **La gramática producto G' : una RCG con $L(G') = L(G) \cap L(A)$.**
- Sobre G' se estudia el vacío y se buscan subclases tratables.

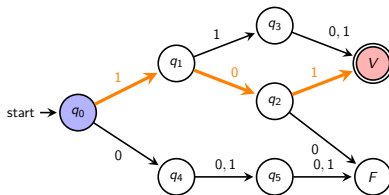
- Rango sobre A : par de estados (q, q') .

Coordinar rangos y estados

- Rango sobre A : par de estados (q, q') .
- Cadena u realiza (q, q') si $\hat{\delta}(q, u) = q'$.

Coordinar rangos y estados

- Rango sobre A : par de estados (q, q') .
- Cadena u realiza (q, q') si $\hat{\delta}(q, u) = q'$.



101 realiza (q_0, V) .

- Predicado indexado $B[\vec{q}]$: un rango por argumento.

- **Predicado indexado** $B[\vec{q}]$: un rango por argumento.
- Ejemplos:

$$S[(q_0, V)], \quad Eq_{x_1}[(q_0, V)], \quad L_1[(q_0, q_1)]$$

- **Predicado indexado** $B[\vec{q}]$: un rango por argumento.
- Ejemplos:

$$S[(q_0, V)], \quad Eq_{x_1}[(q_0, V)], \quad L_1[(q_0, q_1)]$$

- G' refleja qué rango realiza cada argumento.

Cláusulas indexadas

- Cada cláusula de G + asignación de rangos.

Cláusulas indexadas

- Cada cláusula de G + asignación de rangos.
- Se indexa cada predicado por sus rangos.

Cláusulas indexadas

- Cada cláusula de G + asignación de rangos.
- Se indexa cada predicado por sus rangos.

Ejemplo: la cláusula $L_1(1X) \rightarrow L_0(X)$
se indexa como $L_1[(q_0, q_1)](1X) \rightarrow L_0[(q_1, q_1)](X)$.

Cláusulas indexadas

- Cada cláusula de G + asignación de rangos.
- Se indexa cada predicado por sus rangos.

Ejemplo: la cláusula $L_1(1X) \rightarrow L_0(X)$
se indexa como $L_1[(q_0, q_1)](1X) \rightarrow L_0[(q_1, q_1)](X)$.

- Predicado inicial: $S'(X) \rightarrow S[(q_0, q_F)](X)$, $q_F \in F$.

- Cada cláusula de G + asignación de rangos.
- Se indexa cada predicado por sus rangos.

Ejemplo: la cláusula $L_1(1X) \rightarrow L_0(X)$
se indexa como $L_1[(q_0, q_1)](1X) \rightarrow L_0[(q_1, q_1)](X)$.

- Predicado inicial: $S'(X) \rightarrow S[(q_0, q_F)](X)$, $q_F \in F$.
- Lado derecho vacío: solo se indexa el izquierdo, $B(\dots) \rightarrow \varepsilon$ da $B[\vec{q}](\dots) \rightarrow \varepsilon$.

- Cada cláusula de G + asignación de rangos.
- Se indexa cada predicado por sus rangos.

Ejemplo: la cláusula $L_1(1X) \rightarrow L_0(X)$
se indexa como $L_1[(q_0, q_1)](1X) \rightarrow L_0[(q_1, q_1)](X)$.

- Predicado inicial: $S'(X) \rightarrow S[(q_0, q_F)](X)$, $q_F \in F$.
- Lado derecho vacío: solo se indexa el izquierdo, $B(\dots) \rightarrow \varepsilon$ da $B[\vec{q}](\dots) \rightarrow \varepsilon$.
- Ejemplo: $L_0(\varepsilon) \rightarrow \varepsilon$ da $L_0[(q, q)](\varepsilon) \rightarrow \varepsilon$ (una por estado q).

La no linealidad: misma variable, mismo rango

- Una variable repetida es **una sola cadena**.

La no linealidad: misma variable, mismo rango

- Una variable repetida es **una sola cadena**.
- **Condición: igual rango en todas sus ocurrencias.**

La no linealidad: misma variable, mismo rango

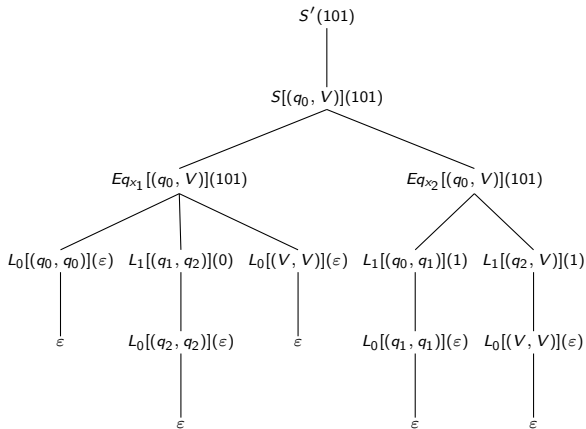
- Una variable repetida es **una sola cadena**.
- **Condición: igual rango en todas sus ocurrencias.**
- Sin ella, 000 entraría: consistente, pero no satisface $x_1 \wedge (x_2 \vee x_1)$.

La no linealidad: misma variable, mismo rango

- Una variable repetida es **una sola cadena**.
- **Condición: igual rango en todas sus ocurrencias.**
- Sin ella, 000 entraría: consistente, pero no satisface $x_1 \wedge (x_2 \vee x_1)$.
- Es lo que separa RCG de sRCG (Bar-Hillel no basta).

101 en la gramática producto G'

Fórmula $x_1 \wedge (x_2 \vee x_1)$: 101 la satisface.



- G' reconoce exactamente $L(G) \cap L(A)$.

- G' reconoce exactamente $L(G) \cap L(A)$.
- **Tamaño exponencial: $|G'| = 2^{O(n \log n)}$.**

- G' reconoce exactamente $L(G) \cap L(A)$.
- **Tamaño exponencial:** $|G'| = 2^{O(n \log n)}$.
- Muchas cláusulas no productivas; limpiarlas queda abierto.

- Llevamos el enfoque de la línea a las RCG: el SAT como vacío de la intersección.

- Llevamos el enfoque de la línea a las RCG: el SAT como vacío de la intersección.
- **Modelamos cualquier fórmula con aridad uno, gracias a la no linealidad; así evitamos que crezca con los cruces, como en SATSRC.**

- Llevamos el enfoque de la línea a las RCG: el SAT como vacío de la intersección.
- **Modelamos cualquier fórmula con aridad uno, gracias a la no linealidad; así evitamos que crezca con los cruces, como en SATSRC.**
- **El precio: ese vacío es NP-completo.**

- Llevamos el enfoque de la línea a las RCG: el SAT como vacío de la intersección.
- **Modelamos cualquier fórmula con aridad uno, gracias a la no linealidad; así evitamos que crezca con los cruces, como en SATSRC.**
- **El precio: ese vacío es NP-completo.**
- Dejamos G' como objeto para buscar subclases tratables.

- Eliminar predicados y cláusulas no productivos.

- Eliminar predicados y cláusulas no productivos.
 - Indexar solo pares de estados útiles (en algún camino aceptante).

- Eliminar predicados y cláusulas no productivos.
 - Indexar solo pares de estados útiles (en algún camino aceptante).
 - Elimina una parte, no todas.

- Eliminar predicados y cláusulas no productivos.
 - Indexar solo pares de estados útiles (en algún camino aceptante).
 - Elimina una parte, no todas.
- Analizar el costo del vacío sobre G' .

- Eliminar predicados y cláusulas no productivos.
 - Indexar solo pares de estados útiles (en algún camino aceptante).
 - Elimina una parte, no todas.
- Analizar el costo del vacío sobre G' .
- Subclases no lineales de RCG con vacío polinomial $\Rightarrow$ resolver más fórmulas de SAT en tiempo polinomial.

- Eliminar predicados y cláusulas no productivos.
 - Indexar solo pares de estados útiles (en algún camino aceptante).
 - Elimina una parte, no todas.
- Analizar el costo del vacío sobre G' .
- Subclases no lineales de RCG con vacío polinomial $\Rightarrow$ resolver más fórmulas de SAT en tiempo polinomial.
- Caracterizar autómatas con pocos caminos aceptantes.

El SAT y el vacío de la intersección de una gramática de concatenación de rango con un lenguaje regular finito

Luis Alejandro Arteaga Morales

Facultad de Matemática y Computación
Universidad de La Habana

2026

Tutores: MSc. Fernando Raúl Rodríguez Flores
Lic. Raudel Alejandro Gómez Molina

- “Su trabajo demuestra que las gramáticas de concatenación de rango permiten modelar la consistencia de *cualquier* fórmula booleana con aridad uno, gracias a la no linealidad, pero reconoce que con ello se pierde la garantía de que el vacío de la intersección sea decidible en tiempo polinomial, garantía que sí existía en SATCF y SATSRC. Siendo así, ¿cuál es el aporte concreto de su tesis respecto a esos antecedentes? Dicho de otro modo: ¿qué problema queda mejor resuelto, o mejor planteado, después de su trabajo que antes de él?”

Punto de partida

Buscar más fórmulas de SAT que se resuelvan en tiempo polinomial, o resolverlas todas, extendiendo SATCF (CFG) y SATSRC (sRCG) a las RCG.

Punto de partida

Buscar más fórmulas de SAT que se resuelvan en tiempo polinomial, o resolverlas todas, extendiendo SATCF (CFG) y SATSRC (sRCG) a las RCG.

- Con sRCG, el vacío crecía exponencialmente con los cruces: subía la aridad.

Punto de partida

Buscar más fórmulas de SAT que se resuelvan en tiempo polinomial, o resolverlas todas, extendiendo SATCF (CFG) y SATSRC (sRCG) a las RCG.

- Con sRCG, el vacío crecía exponencialmente con los cruces: subía la aridez.
- Con RCG, la no linealidad modela **cualquier** fórmula manteniendo la aridez en uno.

Punto de partida

Buscar más fórmulas de SAT que se resuelvan en tiempo polinomial, o resolverlas todas, extendiendo SATCF (CFG) y SATSRC (sRCG) a las RCG.

- Con sRCG, el vacío crecía exponencialmente con los cruces: subía la aridad.
- Con RCG, la no linealidad modela **cualquier** fórmula manteniendo la aridad en uno.

El precio

Al salir de las sRCG, el vacío de la intersección es NP-completo en nuestro caso.

No cierra el camino

Pueden existir subclases no lineales de RCG cuyo vacío de la intersección con un regular finito sea polinomial, aún no descubiertas.

No cierra el camino

Pueden existir subclases no lineales de RCG cuyo vacío de la intersección con un regular finito sea polinomial, aún no descubiertas.

El aporte concreto

Construimos la **gramática producto** G' : un objeto matemático sobre el que estudiar la intersección (su tamaño, qué predicados son productivos, el costo de decidir el vacío, otras propiedades).

No cierra el camino

Pueden existir subclases no lineales de RCG cuyo vacío de la intersección con un regular finito sea polinomial, aún no descubiertas.

El aporte concreto

Construimos la **gramática producto** G' : un objeto matemático sobre el que estudiar la intersección (su tamaño, qué predicados son productivos, el costo de decidir el vacío, otras propiedades).

Por qué es un aporte

La literatura construía la intersección solo hasta sRCG y no manejaba la no linealidad. Este trabajo sí.

- “En la sección 5.3.2 usted exige que la gramática de entrada sea *no borrante*, es decir, que toda variable del lado izquierdo de una cláusula aparezca también en el lado derecho, y resuelve el caso general apelando a que toda RCG admite una equivalente no borrante. Sin embargo, el motivo de fondo es que una variable borrada (presente solo en el lado izquierdo) ocupa una subcadena que *a la gramática le es indiferente, pero que el autómata sí debe leer*. ¿Por qué optó por transformar la gramática a una equivalente no borrante en lugar de tratar el borrado directamente en la construcción? En particular, ¿consideró precomputar un predicado auxiliar que, para cualquier par de estados (q, q') , derive a la cadena vacía si y solo si ese rango es realizable en el autómata, y añadirlo al lado derecho para fijar el rango de la variable borrada? ¿Qué ventajas y costos tiene cada vía?”

El problema

Una variable borrada le es indiferente a G , pero A sí debe leer su subcadena: hay que fijar su rango a uno realizable.

El problema

Una variable borrada le es indiferente a G , pero A sí debe leer su subcadena: hay que fijar su rango a uno realizable.

Dos vías

- **(A) Transformar** G a una equivalente no borrante **(la elegida)**.

El problema

Una variable borrada le es indiferente a G , pero A sí debe leer su subcadena: hay que fijar su rango a uno realizable.

Dos vías

- **(A) Transformar** G a una equivalente no borrante (**la elegida**).
- **(B) Predicado auxiliar**: dada la cláusula borrante $A(XY) \rightarrow B(X)$, reconocer las subcadenas que realizan (q, q') (un subautómata) y engancharlas.

$$A[(q_1, q_2)](XY) \rightarrow B[(q_1, q_3)](X) \text{ Reach}[(q_3, q_2)](Y)$$

Por qué elegimos (A)

- Se apoya en un resultado de Boullier: su correctitud ya está demostrada.
- G_{cons} ya es no borrante.

Por qué elegimos (A)

- Se apoya en un resultado de Boullier: su correctitud ya está demostrada.
- G_{cons} ya es no borrante.

(B) también es válida

- Añade los Reach durante el producto; mismo lenguaje.
- Pero exige rehacer la demostración (sin no borrante).

- “La construcción de la gramática producto exige que la gramática de entrada sea no borrante y no combinatoria, y que todas las ocurrencias de una misma variable en una cláusula reciban el mismo rango. Usted afirma que estas restricciones no descartan ningún lenguaje, pues toda RCG admite una equivalente que las cumple. ¿Podría justificar esa afirmación y precisar el costo de transformar una RCG arbitraria en una equivalente no borrante y no combinatoria? ¿Cómo garantiza, además, que la condición de igualdad de rangos para variables repetidas es suficiente para no reconocer cadenas fuera de la intersección?”

No descarta ningún lenguaje

Boullier (1998), encadenando tres equivalencias:

- toda RCG tiene una equivalente no combinatoria;
- de ahí, una equivalente no borrante;
- se conserva la no linealidad, que es lo que da poder sobre las sRCG.

Lemma 8.7.

1. *For any RCG, there is an equivalent non-combinatorial RCG.*
2. *For any non-combinatorial bottom-up erasing RCG, there is an equivalent non-combinatorial bottom-up non-erasing RCG.*
3. *For any non-combinatorial bottom-up non-erasing top-down erasing RCG, there is an equivalent non-combinatorial non-erasing RCG.*

In other words, the possibilities of combinatorial clauses and erasing clauses do not increase the generative capacity of the grammar. For every RCG, there is an equivalent non-combinatorial non-erasing RCG. The crucial property for RCG's being more powerful than simple RCG is the possible non-linearity of the clauses.

Kallmeyer (2010), *Parsing Beyond Context-Free Grammars*, Lemma 8.7, p. 163.

¿Cuánto cuesta la transformación?

Boullier la da en tres pasos locales (Propiedades 1–3):

Boullier (1998), *A Proposal for a Natural Language Processing Syntactic Backbone*, Properties 1–3, pp. 11–12.

¿Cuánto cuesta la transformación?

Boullier la da en tres pasos locales (Propiedades 1–3):

- No combinatoria: cada cláusula combinatoria $\rightarrow$ tres cláusulas.

Boullier (1998), *A Proposal for a Natural Language Processing Syntactic Backbone*, Properties 1–3, pp. 11–12.

¿Cuánto cuesta la transformación?

Boullier la da en tres pasos locales (Propiedades 1–3):

- No combinatoria: cada cláusula combinatoria $\rightarrow$ tres cláusulas.
- No borrante (lado derecho, bottom-up): el predicado izquierdo gana un argumento con esa variable.

Boullier (1998), *A Proposal for a Natural Language Processing Syntactic Backbone*, Properties 1–3, pp. 11–12.

¿Cuánto cuesta la transformación?

Boullier la da en tres pasos locales (Propiedades 1–3):

- No combinatoria: cada cláusula combinatoria $\rightarrow$ tres cláusulas.
- No borrante (lado derecho, bottom-up): el predicado izquierdo gana un argumento con esa variable.
- No borrante (lado izquierdo, top-down): se añade un predicado *Any* que acepta cualquier cadena.

Boullier (1998), *A Proposal for a Natural Language Processing Syntactic Backbone*, Properties 1–3, pp. 11–12.

Ejemplo 1: no combinatoria

Cláusula combinatoria (XY no es una variable suelta):

$$A(X, Y) \rightarrow B(XY)$$

Ejemplo 1: no combinatoria

Cláusula combinatoria (XY no es una variable suelta):

$$A(X, Y) \rightarrow B(XY)$$

Se reemplaza por tres cláusulas con predicados nuevos c, c' :

$$\begin{aligned} A(W, X, Y) &\rightarrow c(W, X, Y, W) \\ c(W, X, Y, U) &\rightarrow c'(X, Y, Z) \quad B(W, Z) \\ c'(X, Y, XY) &\rightarrow \varepsilon \end{aligned}$$

Ejemplo 1: no combinatoria

Cláusula combinatoria (XY no es una variable suelta):

$$A(X, Y) \rightarrow B(XY)$$

Se reemplaza por tres cláusulas con predicados nuevos c, c' :

$$\begin{aligned} A(W, X, Y) &\rightarrow c(W, X, Y, W) \\ c(W, X, Y, U) &\rightarrow c'(X, Y, Z) \quad B(W, Z) \\ c'(X, Y, XY) &\rightarrow \varepsilon \end{aligned}$$

$c'(X, Y, XY)$ usa la no linealidad para pegar las piezas.

Ejemplo 2: borrado (lado derecho, bottom-up)

Y aparece solo a la derecha:

$$A(X) \rightarrow B(X, Y)$$

Ejemplo 2: borrado (lado derecho, bottom-up)

Y aparece solo a la derecha:

$$A(X) \rightarrow B(X, Y)$$

Se le da a A un argumento nuevo con Y :

$$A(Y, X) \rightarrow B(X, Y)$$

Ejemplo 2: borrado (lado derecho, bottom-up)

Y aparece solo a la derecha:

$$A(X) \rightarrow B(X, Y)$$

Se le da a A un argumento nuevo con Y :

$$A(Y, X) \rightarrow B(X, Y)$$

Ahora Y está en ambos lados.

Ejemplo 3: borrado (lado izquierdo, top-down)

Cláusula borrante (la Y solo está a la izquierda):

$$A(XY) \rightarrow B(X)$$

Ejemplo 3: borrado (lado izquierdo, top-down)

Cláusula borrante (la Y solo está a la izquierda):

$$A(XY) \rightarrow B(X)$$

Se reescribe añadiendo $\text{Any}(Y)$:

$$A(XY) \rightarrow B(X) \text{Any}(Y)$$

$$\text{Any}(aX) \rightarrow \text{Any}(X), \quad \text{Any}(\varepsilon) \rightarrow \varepsilon$$

(a es un terminal cualquiera)

Ejemplo 3: borrado (lado izquierdo, top-down)

Cláusula borrante (la Y solo está a la izquierda):

$$A(XY) \rightarrow B(X)$$

Se reescribe añadiendo $\text{Any}(Y)$:

$$A(XY) \rightarrow B(X) \text{Any}(Y)$$

$$\text{Any}(aX) \rightarrow \text{Any}(X), \quad \text{Any}(\varepsilon) \rightarrow \varepsilon$$

(a es un terminal cualquiera)

Any acepta cualquier cadena: Y se consume sin restringir nada, mismo lenguaje.

Ejemplo 3: borrado (lado izquierdo, top-down)

Cláusula borrante (la Y solo está a la izquierda):

$$A(XY) \rightarrow B(X)$$

Se reescribe añadiendo $\text{Any}(Y)$:

$$A(XY) \rightarrow B(X) \text{Any}(Y)$$

$$\text{Any}(aX) \rightarrow \text{Any}(X), \quad \text{Any}(\varepsilon) \rightarrow \varepsilon$$

(a es un terminal cualquiera)

Any acepta cualquier cadena: Y se consume sin restringir nada, mismo lenguaje.

Es el mismo Any que, al intersectar con A , se vuelve Reach (pregunta 2).

Por qué es polinomial

- Propiedad 1: cada cláusula combinatoria da a lo sumo 3 (factor constante).

Por qué es polinomial

- Propiedad 1: cada cláusula combinatoria da a lo sumo 3 (factor constante).
- Propiedad 2: sube la aridad, no el número de cláusulas.

Por qué es polinomial

- Propiedad 1: cada cláusula combinatoria da a lo sumo 3 (factor constante).
- Propiedad 2: sube la aridez, no el número de cláusulas.
- Propiedad 3: añade $\text{Any}(Y)$ y un bloque fijo de $O(|T|)$ cláusulas.

Por qué es polinomial

- Propiedad 1: cada cláusula combinatoria da a lo sumo 3 (factor constante).
- Propiedad 2: sube la aridad, no el número de cláusulas.
- Propiedad 3: añade $\text{Any}(Y)$ y un bloque fijo de $O(|T|)$ cláusulas.
- G_{cons} ya es no borrante y no combinatoria.

Por qué la igualdad de rangos basta

- Una variable repetida es la misma pieza de w en varios predicados.

Por qué la igualdad de rangos basta

- Una variable repetida es la misma pieza de w en varios predicados.
- En el recorrido real, esa pieza se atraviesa una vez: un solo par de estados.

Por qué la igualdad de rangos basta

- Una variable repetida es la misma pieza de w en varios predicados.
- En el recorrido real, esa pieza se atraviesa una vez: un solo par de estados.
- La condición obliga a que todas sus ocurrencias lleven ese par.

Por qué la igualdad de rangos basta

- Una variable repetida es la misma pieza de w en varios predicados.
- En el recorrido real, esa pieza se atraviesa una vez: un solo par de estados.
- **La condición obliga a que todas sus ocurrencias lleven ese par.**
- Sin ella, los índices describirían un recorrido falso, y G' aceptaría cadenas fuera de la intersección.